

Fable: A ChaCha-Derived Permutation with Single-Pass Authenticated Encryption, Key Commitment and Chunk-Parallel Streaming

Anonymous Submission

Abstract. We present **Fable**, a family of symmetric primitives built on a single new component: **Fable-P**, a 512-bit ARX permutation whose round is the ChaCha double round with round constants injected into two state words. On top of **Fable-P** we define a keyed-duplex authenticated cipher (**Fable-AEAD**; 256-bit key, 192-bit nonce, 256-bit tag, rate $r = 256$, capacity $c = 256$), a chunked large-file mode (**Fable-Stream**, the STREAM construction with 64 KiB chunks) and a sponge hash/XOF. Two parameter sets share everything but the number of data-phase rounds: **Fable** uses P_{12} for initialisation/finalisation and P_8 per data block, **Fable-f** uses P_{12}/P_6 ; the parameter set is bound into the initial state, so the two are domain-separated. The design goals are (i) nonces long enough to be chosen at random, (ii) key commitment without an extra pass, obtained by taking the tag directly from the truncated output of the finalising permutation, (iii) authentication in the same pass as encryption, and (iv) chunk-level parallelism that maps eight independent chunks onto AVX2 lanes.

We report a first-party analysis that is fully reproducible from the accompanying repository: avalanche statistics with 5×10^5 samples per round count; SAT-based (Lipmaa–Moriai) differential trail bounds, including the observation that every rotation-constant choice admits a probability-one differential through one quarter-round, and a rate-restricted model of the duplex data phase that yields a split bound of 2^{-141} on any forgery trail through P_8 ; a differential-linear evaluation calibrated against the ChaCha literature (best key recovery: 7.5 single rounds), giving empirical margins of $2.1\times$ for the **Fable** data phase and $3.2\times$ for initialisation; exact linear-trail optima for one and two rounds and a linear-hull enumeration; rotational-XOR, slide and fixed-point checks of the round constants; a key-commitment reduction (commitment strength $\tau/2$ bits for a τ -bit tag); and multi-user bounds for the STREAM composition. An AVX2 implementation encrypts 1 GiB at 2.00 GB/s (2.44 cycles/byte) single-threaded for **Fable** and 2.53 GB/s for **Fable-f**, on par with or faster than OpenSSL’s ChaCha20-Poly1305 on the same machine, and scales to 11.5 GB/s on eight cores. **Fable** is a research draft: it has received no third-party analysis and must not be used to protect real data.

Keywords: ARX · permutation-based cryptography · keyed duplex · authenticated encryption · key commitment · STREAM · differential cryptanalysis · SAT

1 Introduction

Authenticated encryption in deployed protocols is dominated by two families: AES-GCM, which depends on hardware support for the block cipher and the field multiplication, and ChaCha20-Poly1305 [NL18], which runs on any CPU but pays for authentication with a second pass (Poly1305) over the data and inherits a 96-bit nonce that cannot safely be chosen at random—the extended-nonce variant XChaCha20-Poly1305 [Arc20] repairs the latter with an extra HChaCha20 call. Neither is key-committing [BH22]: for both, an adversary can craft a single ciphertext that decrypts correctly under two different keys,

a property that has led to concrete attacks on message franking, envelope encryption and abuse-reporting systems. Permutation-based designs such as Ascon [DEMS21] and NORX [AJN14] solve the “single pass” part of the problem with the keyed duplex [BDPV12], but they were optimised for lightweight or 64-bit-word platforms respectively, and the commitment question was not a design goal.

Fable is an attempt to combine, in one primitive, the properties we believe a general-purpose file and stream encryption tool needs today:

- **A nonce that can be random.** Fable-AEAD takes a 192-bit nonce. With a 152-bit random file nonce, Fable-Stream stays below a 2^{-73} collision probability for 2^{40} files under one key, and the caller may alternatively use a structured (counter) nonce with zero collision term (§4.10).
- **Key commitment for free.** The tag is the 256-bit rate part of $P_{12}(X)$, where X is the complete internal state before finalisation—a state that contains the key (injected into the capacity), the nonce, the associated data, the message and the parameter-set identifier. We prove (§4.8), in the ideal-permutation model, that any CMT-4 attack reduces to one of three events: a collision on the τ -bit truncation of P_{12} (probability $\leq q^2/2^{\tau+1}$ after q permutation queries), two executions under different keys reaching the same 512-bit state ($\leq q^2/2^{513}$), or a guess of the 256-bit capacity ($\leq q/2^{256}$); the last two are negligible, so a τ -bit tag commits with $\tau/2$ bits of security—128 bits for the default 256-bit tag. This is the CTX idea of Chan and Rogaway [CR22] (commit through a hash of key, nonce, associated data and tag) realised inside the duplex, where the finalising permutation already plays the role of the hash.
- **Authentication in the same pass.** Fable-AEAD is a keyed duplex with $r = c = 256$: every 32-byte block costs one call of the data-phase permutation and nothing else. There is no second pass, no polynomial MAC, and the mode inherits the multi-user bounds of the keyed duplex [JLM14, DMV17] (§4.9).
- **Chunk-level parallelism.** Fable-Stream applies the STREAM construction [HRRV15] to 64 KiB chunks. Chunks are independent AEAD instances, so eight of them are processed simultaneously in the eight 32-bit lanes of an AVX2 register file, with the sixteen state words kept in transposed layout for the whole AEAD; decryption is online and truncation, reordering and cross-file splicing are rejected by the STREAM reduction.

Why a new permutation. The permutation Fable-P is deliberately conservative: its round function is exactly the ChaCha double round (column step followed by diagonal step, quarter-round rotations (16, 12, 8, 7)) [Ber08a], so that nearly two decades of differential-linear cryptanalysis of ChaCha [AFK⁺08, BLT20, BBC⁺22, CS21, DGSS22, DGM23, WLHL23, Dey24, SDSM25, FGT25b, FGT25a] transfers, subject to the calibration in §4. The one structural change is required by the change of use: ChaCha is a PRF whose fixed input constants break symmetry, whereas a sponge permutation receives attacker-influenced input in every word and must break symmetry *inside* the round. Fable-P therefore XORs a round constant into two words, $S[0]$ and $S[5]$, chosen so that they lie in different columns, different rows and on the same diagonal. We show in §3.2 that without the constants the “four identical columns” subspace of 2^{128} states is invariant under every round, and that the constants leave that subspace in every one of the twelve rounds.

Security argument. ARX permutations have no provable active-S-box bound; Fable-P, like ChaCha, even has probability-one differentials through a single quarter-round (for every choice of rotation constants, §4.2). We therefore make no “no characteristic above

2⁻¹²⁸ in n rounds” claim. Instead the round counts are set by an empirical margin against the strongest published attack family on the same round function: with key recovery on 7.5 ChaCha single rounds as the yardstick, the Fable data phase ($P_8 = 16$ single rounds) has a 2.1 $\times$ margin, initialisation ($P_{12} = 24$ single rounds) 3.2 $\times$, and Fable-f ($P_6 = 12$ single rounds) 1.6 $\times$, the margin of ChaCha12 as used in Adiantum [CB18]. ChaCha20’s own round count was fixed by Bernstein at design time [Ber08b, Ber08a]; the question of how much margin is enough has since been debated explicitly [Aum20], and we make our choice and its yardstick visible rather than settle that debate. Where exact statements are possible we make them: SAT-proven trail bounds, an exact key-commitment reduction, and mode-level bounds in the ideal-permutation model.

Relation to existing constructions. Each of the four goals above is met by some deployed or standardised scheme; the point of Fable is to meet all four with one permutation and one pass. XChaCha20-Poly1305 [Arc20] provides a 192-bit nonce and portable performance, and becomes key-committing when wrapped with CTX [CR22] at the cost of an additional hash of the key, nonce, associated data and tag; authentication remains a second pass, and there is no built-in chunked mode. AES-GCM-SIV [GLL19] offers nonce-misuse resistance, which Fable does not claim, but its nonce is 96 bits, it is not key-committing, and its speed depends on AES and carry-less multiplication hardware. Ascon-128a [DEMS21] is a keyed duplex with single-pass authentication and a 128-bit key; it targets constrained platforms, its nonce is 128 bits, and its tag is XORed with the key at finalisation, which is not the direct truncated-permutation output that our commitment reduction relies on. AEGIS [WP14, DL25] reaches very high throughput with AES round instructions and has a 256-bit-nonce variant; it is not designed for key commitment, and it relies on hardware AES. Fable sits in the software-only, ARX corner of this space: a random-safe nonce, single-pass authentication, commitment from the tag itself, and chunk-level SIMD parallelism, at the price of a permutation that—unlike AES or ChaCha20—has not yet received third-party analysis.

Contributions.

1. The specification of Fable-P, Fable-AEAD, Fable-Stream and Fable-Hash with two domain-separated parameter sets and test vectors (§2).
2. A design rationale that records, for each choice, the experiment that motivated it (§3).
3. A reproducible first-party analysis (§4): avalanche with 5×10^5 states per round count; SAT differential bounds in the unconstrained model (1 round: 2^{-2} exact; 2 rounds: $\leq 2^{-33}$) and in the rate-restricted duplex model (1 round: 2^{-22} exact; 2 rounds: $\leq 2^{-42}$; no internal collision within half a round), combined into a split bound of 2^{-141} for any forgery trail through P_8 and 2^{-108} through P_6 ; a rotation-constant screening over all 32^4 choices for the single-quarter-round metric and 254,576 choices for the one-round metric, showing that neither metric can separate them; a differential-linear evaluation with 2^{22} samples per bit pair showing that the round constants do not change the bias envelope relative to the ChaCha round structure; exact linear-trail optima (1 round 2^{-1} , 2 rounds 2^{-20}) and a hull enumeration (no hull effect in 1 round; factors 3–9 in 2 rounds depending on the mask pair); rotational-XOR bounds ($\leq 2^{-419}$ for P_6); slide and fixed-point checks; the key-commitment reduction; and multi-user STREAM bounds.
4. Portable C and AVX2 implementations, byte-exact against the Python reference, with throughput measurements against OpenSSL on the same machine (§5).

Status. Fable is a research draft (specification v0.4). Every number in this paper is taken from the analysis log of the public repository, each with the command that regenerates it and the raw data file it was computed from; the log also lists what has *not* been done (§6). Until independent cryptanalysis has been published, Fable must not be used to protect real data.

2 Specification

This section is a transcription of specification v0.4 (**Fable-spec-v0.4.md** in the repository); the Python reference **fable.py** and the C implementation are the normative tie-breakers and agree bit for bit on all test vectors.

2.1 Notation

All words are 32-bit unsigned integers; byte strings are converted to words little-endian. $+$ denotes addition modulo 2^{32} , $\oplus$ XOR, and $x \lll n$ rotation of a 32-bit word to the left by n positions. The state S consists of sixteen words $S[0..15]$, viewed as a 4×4 matrix in row-major order:

$$\begin{array}{cccc} S[0] & S[1] & S[2] & S[3] \\ S[4] & S[5] & S[6] & S[7] \\ S[8] & S[9] & S[10] & S[11] \\ S[12] & S[13] & S[14] & S[15] \end{array}$$

The *rate* is $R = S[0..7]$ (256 bits) and the *capacity* is $C = S[8..15]$ (256 bits). P_n denotes n rounds of Fable-P. $\text{pad}(x)$ is the 10^* padding of a byte string x with $|x| < 32$ bytes to 32 bytes: append the byte $0x80$, then zero bytes.

2.2 The permutation Fable-P

Quarter-round. $Q(a, b, c, d)$ updates four words in place with the rotation constants $(r_0, r_1, r_2, r_3) = (16, 12, 8, 7)$:

$$\begin{array}{l} a += b; d \oplus= a; d \lll r_0; \quad c += d; b \oplus= c; b \lll r_1; \\ a += b; d \oplus= a; d \lll r_2; \quad c += d; b \oplus= c; b \lll r_3. \end{array}$$

This is the ChaCha quarter-round.

Round. Round i ($0 \leq i \leq 11$) consists of constant injection, a column step and a diagonal step:

$$\begin{array}{l} S[0] \oplus= \text{RC}[i]; \quad S[5] \oplus= \text{RC}[i] \lll 16; \\ Q(S[0], S[4], S[8], S[12]); \quad Q(S[1], S[5], S[9], S[13]); \\ Q(S[2], S[6], S[10], S[14]); \quad Q(S[3], S[7], S[11], S[15]); \\ Q(S[0], S[5], S[10], S[15]); \quad Q(S[1], S[6], S[11], S[12]); \\ Q(S[2], S[7], S[8], S[13]); \quad Q(S[3], S[4], S[9], S[14]). \end{array}$$

The column and diagonal steps together are the ChaCha double round.

Round constants. $\text{RC}[i]$ is the first 32 bits of the fractional part of $\ln p_i$, where p_i is the $(i+1)$ -th prime $(2, 3, 5, \dots, 37)$; the twelve values are listed in Appendix A. P_{12} uses $\text{RC}[0..11]$ in order. A reduced permutation P_n with $n < 12$ uses the *last* n constants, $\text{RC}[12 - n..11]$: P_8 uses $\text{RC}[4..11]$ and P_6 uses $\text{RC}[6..11]$, so that the light permutation is a suffix of the heavy one.

2.3 Parameter sets

Two parameter sets share the permutation, the constants, the mode and the key format and differ only in the number of rounds d of the data-phase permutation P_d and in the constant IV_1 that encodes it (Table 1). Because IV_1 enters the initial state, the two sets are domain-separated: outputs under the same key are unrelated.

Table 1: Parameter sets. IV_1 packs version 0x01, key bytes 0x20, rate bytes 0x20 and a rounds byte $(12 \ll 4) \mid d$.

Set	init/final	data phase	IV_1	Intended use
Fable (default)	P_{12}	P_8	0x012020C8	general purpose
Fable-f	P_{12}	P_6	0x012020C6	throughput first (ChaCha12 margin)

2.4 Fable-AEAD

Inputs: key K (256 bits, words $K[0..7]$), nonce N (192 bits, words $N[0..5]$), associated data A and message M of arbitrary byte length (a single message is limited to 2^{64} bytes; larger objects use Fable-Stream). Output: ciphertext C with $|C| = |M|$ and a 256-bit tag T ; T may be truncated by the application to no fewer than 128 bits. $IV_0 = 0x4661626C$ (ASCII “Fabl”).

Algorithm 1 Fable-AEAD encryption, parameter set with data-phase rounds d and constant IV_1

Require: $K \in \{0, 1\}^{256}$, $N \in \{0, 1\}^{192}$, $A, M \in \{0, 1\}^{8*}$

Ensure: $C \in \{0, 1\}^{|M|}$, $T \in \{0, 1\}^{256}$

```

1:  $S \leftarrow IV_0 \parallel IV_1 \parallel K[0..7] \parallel N[0..5]$   $\triangleright S[0]=IV_0, S[1]=IV_1, S[2..9]=K, S[10..15]=N$ 
2:  $S \leftarrow P_{12}(S)$ 
3:  $S[8..15] \leftarrow S[8..15] \oplus K[0..7]$   $\triangleright$  key re-injected into the capacity
4: if  $|A| > 0$  then
5:   split  $A$  into 32-byte blocks  $A_0, \dots, A_{a-1}$  with  $|A_{a-1}| < 32$  (the last block may be empty)
6:   for  $j = 0$  to  $a - 2$  do
7:      $S[0..7] \leftarrow S[0..7] \oplus A_j$ ;  $S \leftarrow P_d(S)$ 
8:   end for
9:    $S[0..7] \leftarrow S[0..7] \oplus \text{pad}(A_{a-1})$ ;  $S \leftarrow P_d(S)$ 
10: end if
11:  $S[15] \leftarrow S[15] \oplus 0x80000000$   $\triangleright$  domain separation: end of associated data
12: split  $M$  into 32-byte blocks  $M_0, \dots, M_{m-1}$  with  $|M_{m-1}| < 32$  (the last block may be empty)
13: for  $j = 0$  to  $m - 2$  do
14:    $S[0..7] \leftarrow S[0..7] \oplus M_j$ ;  $C_j \leftarrow S[0..7]$ ;  $S \leftarrow P_d(S)$ 
15: end for
16:  $S[0..7] \leftarrow S[0..7] \oplus \text{pad}(M_{m-1})$ ;  $C_{m-1} \leftarrow$  first  $|M_{m-1}|$  bytes of  $S[0..7]$ 
17:  $S[8..15] \leftarrow S[8..15] \oplus K[0..7]$   $\triangleright$  finalisation
18:  $S \leftarrow P_{12}(S)$ 
19:  $T \leftarrow S[0..7]$   $\triangleright$  the tag is the rate itself; it is not XORed with the key
20: return  $C_0 \parallel \dots \parallel C_{m-1}, T$ 

```

Algorithm 1 gives encryption. Note that the message is always terminated by a padded block, even when $|M|$ is a multiple of 32 or M is empty, whereas an empty A is not absorbed at all; the bit set in $S[15]$ after the associated data separates the two phases.

Decryption is the mirror image: each full ciphertext block C_j is XORed with the rate to recover M_j and then *replaces* the rate before P_d ; for the last block the plaintext is recovered from the truncated keystream and $\text{pad}(M_{m-1})$ is XORed into the rate. The received tag is compared with the recomputed one in constant time; no plaintext is released on failure.

2.5 Fable-Stream

Fable-Stream is the STREAM construction [HRRV15] over **Fable-AEAD**. Parameters: a chunk size L (default 65536 bytes, recorded in the header) and a 152-bit file nonce N_f . The file header is

$$H = \text{"FBLs"} \parallel 0\text{x}01 \parallel \text{LE}_{32}(L) \parallel N_f \quad (28 \text{ bytes}),$$

and the plaintext is split into $n = \lfloor |M|/L \rfloor + 1$ chunks $M_0, \dots, M_{n-1}$, of which the last has fewer than L bytes and may be empty. Chunk i is encrypted with **Fable-AEAD** under the chunk nonce

$$N_i = N_f \parallel \text{LE}_{32}(i) \parallel \text{flag}_i, \quad \text{flag}_i = 0\text{x}01 \text{ if } i = n - 1 \text{ else } 0\text{x}00,$$

with associated data H , producing $C_i \parallel T_i$; the output is $H \parallel C_0 \parallel T_0 \parallel \dots \parallel C_{n-1} \parallel T_{n-1}$. The counter allows 2^{32} chunks (256 TiB at the default chunk size). A final chunk with $\text{flag} = 0\text{x}01$ is always present, so truncation at a chunk boundary is detected; reordering is detected through the counter and cross-file splicing through N_f and H . Decryption verifies chunks in order and stops at the first failure; chunks released before that point are authentic (the online property of STREAM). N_f is either generated uniformly at random or supplied by the caller as a structured value that is unique per key (§4.10).

2.6 Fable-Hash and Fable-XOF

The hash is a plain sponge with $r = c = 256$ and P_{12} throughout. The initial state is $\text{IV}_0 \parallel 0\text{x}01202000 \parallel 0^{14 \times 32}$ (the rounds byte of IV_1 is zero, which separates hashing from every AEAD parameter set). Absorption XORs 32-byte blocks into $S[0..7]$ with a P_{12} after each, the last block 10^* -padded as above; squeezing outputs $S[0..7]$ and applies P_{12} between output blocks. **Fable-Hash-256** is the first 32 bytes of the XOF. Claimed: 128-bit security against all attacks (collision, preimage and second preimage), the flat sponge claim for $c = 256$; cf. SHAKE128.¹

2.7 Test vectors

Appendix B lists P_{12} and P_8 applied to the all-zero state and one AEAD vector per parameter set. The full set (five AEAD vectors per parameter set, two hash vectors, both permutation vectors) is `test_vectors_v0.3.json` in the repository; the **Fable-f** vectors coincide with the v0.1 vectors, which were generated before the parameter set existed.

3 Design Rationale

Every choice below is tied to an experiment or a proof in §4; the last sentence of each subsection names it.

¹Specification versions v0.1–v0.4 claimed 256-bit preimage resistance for the XOF. That claim was wrong: with $c = 256$ a generic internal-collision (meet-in-the-middle) attack finds preimages in $2^{c/2} = 2^{128}$ work regardless of the output length. Version v0.4.1 and this paper correct it.

3.1 ARX and the ChaCha double round

Fable-P is an ARX permutation on sixteen 32-bit words for three reasons. First, the operations map one-to-one onto SIMD lanes: a 512-bit state is half an AVX2 register file, eight independent states fit in the eight 32-bit lanes of the sixteen `ymm` registers, and the rotations by 16 and 8 are byte shuffles. Second, ARX has no table look-ups and no data-dependent branches, so constant-time execution is the default rather than an engineering effort. Third, by taking the round function to be exactly the ChaCha double round—column step, diagonal step, quarter-round rotations (16, 12, 8, 7)—we inherit a body of published cryptanalysis that is unusually deep for a symmetric primitive: differential-linear attacks with probabilistic neutral bits have been refined continuously since 2008 [AFK⁺08, BLT20, BBC⁺22, CS21, DGSS22, DGM23, WLHL23, Dey24, SDSM25, FGT25b, FGT25a], and the best of them is the yardstick for our round counts. The price is well known: an ARX round has no provable lower bound on the number of active S-boxes, so no “ n rounds have no characteristic above 2^{-128} ” statement can be made the way it is for an SPN such as Ascon’s S-box layer [DEMS21]. We accepted this trade because, on CPUs without AES instructions, a wide-trail SPN at the same throughput would need a bitsliced S-box layer, and because the questions an ARX design leaves open are precisely the ones the ChaCha literature has spent two decades sharpening. The calibration of that literature to Fable-P is in §4.4, and the exact statements we can make (SAT trail bounds) are in §4.2 and §4.5.

3.2 Round constants and the column-symmetric subspace

ChaCha is a PRF: its input contains four fixed constant words, and every round acts on a state that the attacker can only influence through the key, nonce and counter positions. A sponge permutation enjoys no such protection—after the first block the whole state is a function of attacker-chosen data—so symmetry must be broken inside the round. Consider the round function F_0 with all constants set to zero, i.e. the bare ChaCha double round.

Lemma 1 (column-symmetric subspace). *Let $\mathcal{V} = \{S : S[4j + k] = S[4j] \text{ for all } j, k \in \{0, 1, 2, 3\}\}$ be the set of states whose four columns are identical ($|\mathcal{V}| = 2^{128}$). Then $F_0(\mathcal{V}) = \mathcal{V}$.*

Proof. For $S \in \mathcal{V}$ the four column quarter-rounds

$$Q(S[j], S[4+j], S[8+j], S[12+j]), \quad j = 0, \dots, 3,$$

receive the same input $(S[0], S[4], S[8], S[12])$, hence produce the same output, so the state after the column step is again in $\mathcal{V}$. For such a state the four diagonal quarter-rounds

$$\begin{aligned} &Q(S[0], S[5], S[10], S[15]), \quad Q(S[1], S[6], S[11], S[12]), \\ &Q(S[2], S[7], S[8], S[13]), \quad Q(S[3], S[4], S[9], S[14]) \end{aligned}$$

also receive identical inputs—each diagonal picks one word from each row, and all words of a row are equal—so the diagonal step preserves $\mathcal{V}$ as well. $\square$

The all-zero state is a fixed point of F_0 (it lies in $\mathcal{V}$ and $Q(0, 0, 0, 0) = 0$), and $\mathcal{V}$ is the only “large” symmetric class we found to be invariant: the classes “four identical rows” and “all sixteen words equal” are left after one round. A constant XORed into a single word $S[0]$ already makes column 0 differ from columns 1–3; Fable-P injects into two words, $S[0]$ and $S[5]$, which lie in different columns (0 and 1), different rows (0 and 1) and on the same diagonal, so that in one round the constant enters two of the four column quarter-rounds and one diagonal quarter-round, and the two injections never cancel: $\text{RC}[i] \oplus (\text{RC}[i] \lll 16) \neq 0$ for all i and $\text{RC}[i] \neq \text{RC}[j] \lll 16$ for all i, j .

The constants themselves are the fractional parts of $\ln p_i$ for the first twelve primes. We chose the natural logarithm rather than the square or cube roots used by SHA-2 so that no bit pattern is shared with another standard and no “reused constant” question arises; the resulting words are pairwise distinct, none is a rotation of another, and their minimum pairwise Hamming distance is 9. A reduced permutation uses the *last* n constants, so that P_8 and P_6 are suffixes of P_{12} , as in Ascon [DEMS21]: any analysis of the heavy permutation restricted to its last rounds is directly an analysis of the light one, and the suffix sequence RC[6..11] does not recur as a window elsewhere in the table, which is what a slide attack between the two permutations would need.

The corresponding checks—the invariance of $\mathcal{V}$ under F_0 for 1–12 rounds, its destruction in every one of the twelve rounds on 10^4 random column-symmetric states, the absence of structured fixed points and of equal or rotated constants—are in §4.7; the rotational-XOR cost the constants impose is in §4.6.

3.3 Rotation constants

The rotation amounts (16, 12, 8, 7) are inherited from ChaCha, and the specification originally marked them provisional pending a screening of all 32^4 alternatives by differential trail weight. The screening (§4.3) found that the natural metrics degenerate in order: the best trail through a single quarter-round has weight 0 for *every* one of the 2^{20} choices, because a difference confined to the most significant bit passes all four additions for free (Lemma 2); the best one-round trail of Fable-P has weight exactly 2 for every sampled choice without a zero rotation (201,422 of 254,576); and the best 1.5-round trail saturates at weight 12 for 67% of a 2000-choice sample, with no choice above 12. On the only metric that separated candidates—one-round bit-level diffusion—the ChaCha constants rank in the middle of the top class, and none of the 20 fastest-diffusing candidates beat them on the two-round trail bound (all ≥ 25 , the baseline ≥ 33). Since no differential argument favours a change, and the ChaCha constants carry the analysis heritage of §3.1, they were kept; the fact that 16 and 8 are byte shuffles on AVX2 and NEON is a welcome side effect. The full ranking tables are in §4.3.

3.4 Round counts

Because no trail-based argument reaches 2^{-128} for a full data-phase permutation (the best we prove is a 2^{-141} split bound for the forgery model of P_8 , §4.2), the round counts are set by an empirical margin: the ratio between the number of ChaCha single rounds in a permutation and the number reached by the strongest published key-recovery attack on ChaCha, which is 7.5 as of 2025 [FGT25b, Dey24, SDSM25]. The default Fable data phase P_8 has 16 single rounds and a margin of $2.1\times$; initialisation and finalisation P_{12} have 24 and $3.2\times$, more than ChaCha20’s own $20/7.5 = 2.7\times$. Fable-f keeps P_6 (12 single rounds, $1.6\times$) for throughput-bound settings that accept the margin of ChaCha12 [CB18]; it was the only data-phase permutation in versions v0.1–v0.2, and was demoted to an optional set when the $1.6\times$ margin was judged too close to the $1.5\times$ floor the specification sets for itself. A five-round data phase (10 single rounds, $1.33\times$) was considered for speed and removed for the same reason. The comparison is meaningful only if the round constants do not make Fable-P’s differential-linear behaviour different from ChaCha’s; §4.4 measures this directly and finds the bias envelopes identical.

3.5 Rate and capacity

With $b = 512$, the split $r = c = 256$ gives the keyed duplex its “beyond $2^{c/2}$ ” regime [JLM14]: the dominant multi-user term is $MN/2^c$, where M is the total number of processed blocks and N the attacker’s offline permutation evaluations [DMV17]. At $c = 256$ this term

is 2^{-37} even for $\mu = 2^{32}$ users, 2^{64} bytes per user ($M \approx 2^{91}$) and $N = 2^{128}$, and 2^{-69} for a single user. A 320-bit rate ($c = 192$) would raise throughput by 25 % but makes the same term 2^{27} in the multi-user setting and 2^{-5} even for one user at the data limit; such a variant could only be offered with a single-user, data-limited claim, so it is not a general-purpose parameter set. The bound and its evaluation are in §4.9.

3.6 The tag is the rate, not the rate XOR the key

Fable-AEAD outputs $T = S[0..7]$ after the finalising P_{12} and does not XOR the key into it. The key has already been injected into the capacity before that permutation, so the tag is the τ -bit truncation of P_{12} applied to a state that is a function of the key, the nonce, the associated data, the message and the parameter-set identifier. Any two inputs that produce the same tag therefore either produce the same 512-bit state with a fixed key difference (an event of probability at most $q^2/2^{513}$ after q permutation queries) or a collision of a truncated permutation, and the commitment strength is $\tau/2$ bits for the cost of nothing.

The key is injected into the capacity twice more, after initialisation and before finalisation, as in Ascon [DEMS21]. The first re-injection means that an adversary who recovers the full state during the data phase still does not learn the key; the second means that even a full state recovered after the last data block does not allow forging the tag, since the finalising permutation is applied to a state that depends on the key once again. Both are cheap (eight XORs each) and neither interferes with the commitment argument, because the key is part of the state that the tag commits to either way. XORing the key into the tag, as Ascon does, would not obviously weaken this—the adversary would then need $F(X) \oplus F(X') = K \oplus K'$ —but it would make the reduction depend on a relation between the key and the tag rather than on the collision resistance of a public function alone, and we preferred the argument whose only assumption is that P_{12} has no collision shortcut. The reduction is Theorem 1 in §4.8.

3.7 Nonce length, file nonce, chunk size and STREAM

The AEAD nonce is 192 bits so that Fable-Stream can spend 32 bits on a chunk counter and 8 bits on the end-of-file flag and still leave 152 random bits for the file nonce N_f . For F files under one key the probability that two random file nonces coincide is at most $F^2/2^{153}$: 2^{-73} at $F = 2^{40}$ and 2^{-57} in the degenerate case of 2^{48} single-chunk files that exhausts the 2^{64} -byte budget. This is the largest term in the concrete security bound (§4.9), which is why the specification also admits a caller-supplied structured N_f with zero collision term. The default chunk of 64 KiB is a compromise between overhead and granularity: the 32-byte tag per chunk is 0.05 % of the data, the two P_{12} calls per chunk add about 0.2 % to the round count of the 2048 data-phase permutations, and 64 KiB is large enough for the eight-lane AVX2 implementation to amortise the transposition of state and data. STREAM [HRRV15] was chosen over a single long duplex stream because it gives, at that cost, independent chunks (thread- and lane-level parallelism, random access on decryption—with the truncation guarantee requiring the final chunk to be checked), online decryption in which every released chunk is already authenticated, and a tight reduction to the nonce-based AEAD security of Fable-AEAD that covers truncation, reordering and cross-file splicing. The STREAM reduction and the nonce term are in §4.9 and §4.10; the throughput of the eight-lane implementation is in §5.

4 Security Analysis

All results in this section are first-party and reproducible: every number is taken from the analysis log of the repository (`ANALYSIS-LOG.md`), which records for each the command that regenerates it and the raw data file it was computed from; we cite those files inline. Three distinctions are kept strictly throughout. A *trail* (characteristic) fixes the difference or mask at every intermediate step and its probability or correlation is computed under the usual independence assumption; a *differential* (resp. linear hull) aggregates all trails with the same input and output and can only have larger probability (resp. a different correlation). A statement is *exact* when the SAT solver has both found a trail of weight W and proved that none of weight $W - 1$ exists; it is a *SAT lower bound* on the weight (hence an upper bound on the trail probability) when the solver proved “no trail of weight $\leq W - 1$ ” and was stopped before finding one; it is a *split bound* when it is the sum of bounds for consecutive segments (§4.2). Bounds on weights are lower bounds; the corresponding bounds on probabilities are upper bounds.

4.1 Avalanche and diffusion

Model. For n rounds of Fable-P, each of the 512 input bits is flipped in $N = 5 \times 10^5$ uniformly random states and, for each of the 512×512 (input bit, output bit) pairs, the fraction p of states in which the output bit changed is recorded (`c/avalanche.c`, data `data/avalanche/av_r{1..6}.u32`). Under the null hypothesis $p = 1/2$ the estimator has standard deviation $\sigma = 0.5/\sqrt{N} = 7.1 \times 10^{-4}$, and the expected maximum of 2^{18} independent Gaussian deviations is about $\sigma\sqrt{2 \ln 2^{18}} \approx 5\sigma \approx 0.0035$. A structural bias of 0.005 would therefore stand out at 7σ , twice the noise ceiling.

Table 2: Avalanche after n rounds, 5×10^5 states per round count. “Pairs” counts the 512×512 input/output bit pairs.

n	mean p	$\max p - \frac{1}{2} $	pairs > 0.02	pairs > 0.01	pairs > 0.005	never-flipping pairs
1	0.35588	0.50000	188107	199886	206640	1881
2	0.50000	0.00320	0	0	0	0
3	0.50000	0.00299	0	0	0	0
4	0.50000	0.00320	0	0	0	0
5	0.50000	0.00327	0	0	0	0
6	0.50000	0.00356	0	0	0	0

Result. Table 2: after one round every word depends on every word but 57% of the bit pairs are still deterministic and 1881 pairs never flip; from two rounds on the maximum deviation (0.0030–0.0036) equals the noise ceiling and the whole histogram lies in the first 0.005 bin. Two rounds achieve full bit-level avalanche with no structural bias at the 0.005 level. A consequence for the design process (§4.3) is that two-round avalanche cannot discriminate between rotation constants; it saturates for all of them.

4.2 Differential trails

Model. XOR differences propagate through XOR and rotation deterministically and through modular addition with the Lipmaa–Moriai conditions [LM02]: $(\alpha, \beta \rightarrow \gamma)$ is a valid differential of $z = x + y$ iff $\alpha_0 \oplus \beta_0 \oplus \gamma_0 = 0$ and, for $0 \leq i \leq 30$, $\alpha_i = \beta_i = \gamma_i$ implies $\alpha_{i+1} \oplus \beta_{i+1} \oplus \gamma_{i+1} = \alpha_i$; its weight is the number of positions $i \leq 30$ with $\neg(\alpha_i = \beta_i = \gamma_i)$ and its probability $2^{-\text{weight}}$. Round constants are transparent to XOR differences. The permutation is encoded in CNF with one weight literal per addition bit (Appendix C), the total weight is bounded with an incremental totalizer, and the minimum is found

by increasing the bound until the instance is satisfiable (CaDiCaL [BFFH20] through PySAT [IMMS18]); this is the approach Mouha and Preneel applied to Salsa20 [MP13], with a SAT rather than an SMT back end. Found trails are checked empirically by sampling through the C permutation (`sat/verify_trail.py`).

Lemma 2 (probability-one quarter-round differential). *For every choice of rotation constants (r_0, r_1, r_2, r_3) , the quarter-round Q has a differential of probability one with a nonzero input difference, namely $(\Delta a, \Delta b, \Delta c, \Delta d) = (0, \text{MSB}, 0, \text{MSB} \oplus \text{bit}(31 - r_0)) \rightarrow (\text{MSB}, \text{bit}(r_3 - 1), \text{MSB}, 0)$, where $\text{MSB} = \text{bit}(31)$ and $\text{bit}(j) = 2^j$.*

Proof. A difference confined to the most significant bit passes through a modular addition with probability one, because no carry propagates out of bit 31. In $a += b$ the difference MSB of b becomes $\Delta a = \text{MSB}$. In $d \oplus = a$ the MSB of Δd cancels and $\text{bit}(31 - r_0)$ remains, which $d \lll r_0$ moves to the MSB. Then $c += d$ gives $\Delta c = \text{MSB}$ with probability one, $b \oplus = c$ cancels Δb to zero, so $b \lll r_1$ carries no difference; $a += b$ leaves $\Delta a = \text{MSB}$; $d \oplus = a$ cancels Δd to zero; $c += d$ leaves $\Delta c = \text{MSB}$; finally $b \oplus = c$ sets $\Delta b = \text{MSB}$ and $b \lll r_3$ moves it to $\text{bit}(r_3 - 1)$. No addition ever sees a difference below bit 31, so the total weight is 0. $\square$

For $(16, 12, 8, 7)$ the SAT search returns exactly this trail and 10^5 random samples confirm it; the search over all 32^4 rotation choices (§4.3) returns weight 0 for each, as the lemma predicts. This is the known behaviour of the ChaCha quarter-round, inherited unchanged.

Unconstrained model. Table 3 gives the minimum trail weight of P_n when the input difference may be anywhere in the 512-bit state. The one-round optimum (weight 2, 2^{-2} , verified: 2×10^5 samples hit at rate 0.2498) places MSB-derived differences in two columns, cancels them to two MSBs in the column step and pays 2 in the diagonal step. The two-round search proved “no trail of weight ≤ 30 ” after 9.4 CPU-hours, was resumed from that bound and proved “ ≤ 32 ” unsatisfiable (the last level alone took 20 729 s) before being stopped; the optimum is ≥ 33 , no trail is known. Three and four rounds are split bounds only.

Table 3: Minimum differential trail weight of Fable-P, unconstrained input difference (`data/diff/perm_r*.json`, `perm_r2_baseline_cadical.log`).

rounds	weight	trail probability	status
$\frac{1}{2}$ (one Q)	0	1	exact (Lemma 2)
1	2	2^{-2}	exact, verified
2	≥ 33	$\leq 2^{-33}$	SAT lower bound (stopped)
3	≥ 35	$\leq 2^{-35}$	split bound $2 + 33$
4	≥ 66	$\leq 2^{-66}$	split bound $33 + 33$

Rate-restricted models. In the data phase of the keyed duplex the adversary injects differences only through the 256-bit rate and observes only the rate. We therefore add three optional constraints to the SAT model:

- **rate-in:** the input difference is zero on $S[8..15]$ (the attacker controls only the message block);
- **cap-out-zero:** the output difference is zero on $S[8..15]$ (an *internal collision*: the rate difference can be cancelled by the next message block, the capacities coincide, and every later tag collides—the forgery model);

- **rate-out-nonzero**: the output difference is nonzero on $S[0..7]$ (a difference observable in the keystream).

Table 4 summarises. The rate-restricted one-round optimum is 22, twenty bits more than the unconstrained one, because Lemma 2 needs a difference in the capacity word d of the column quarter-round; the optimal trail places MSB-derived differences in $S[0]$ and $S[4]$ of one column. Its probability was checked with 2×10^7 random pairs: 3 hits against $2^{-22} \cdot 2 \times 10^7 = 4.8$ expected, consistent under Poisson statistics. An internal collision within half a round (column step only) is impossible: the instance is unsatisfiable up to weight 160, since the column step necessarily pushes a rate difference into the two capacity words of its column. With any input allowed, a one-round trail into a zero capacity difference costs only 6, so the **cap-out-zero** constraint is cheap on its own; what is expensive is confining the *input* to the rate.

Table 4: Minimum trail weight under the duplex data-phase constraints (data/diff/perm_r*_rate*.json, *.log). “imp.” = unsatisfiable up to weight 160.

constraints	$\frac{1}{2}$ round	1 round	2 rounds	3 rounds
none	0	2 (exact)	≥ 33	≥ 35 (split)
rate-in	2 (exact)	22 (exact, verified)	≥ 42	≥ 44 (split; SAT reached 43)
rate-in + rate-out-nonzero	–	22 (exact)	–	–
rate-in + cap-out-zero	imp.	≥ 43	≥ 41	–
cap-out-zero only	–	6 (exact, verified)	–	–

Split bounds. Cut an R -round trail into consecutive segments. The differences at the cuts are left unconstrained, so the weight of the whole trail is at least the sum, over segments, of the minimum weight of that segment under only the constraints at its own two ends. A segment bound proved under fewer constraints remains valid under more, so the last segment of a forgery trail may use the unconstrained bound. For P_8 in the forgery model (rate-in at the start, cap-out-zero at the end) the candidates are listed in Table 5; the best is four two-round segments.

Table 5: Split bounds for a forgery trail through P_8 . Segment bounds from Tables 3 and 4.

segments (rounds: bound used)	total weight
2 (rate-in ≥ 42) + 2 (none ≥ 33) + 2 (none ≥ 33) + 2 (none ≥ 33)	141
1 (rate-in 22) + 2 (33) + 2 (33) + 2 (33) + 1 (none 2)	123
2 (rate-in ≥ 42) + 2 (33) + 2 (33) + 1 (none 2) + 1 (cap-out-zero 6)	116
3 (rate-in ≥ 44) + 2 (33) + 2 (33) + 1 (none 2)	112

Result. Every differential *trail* usable for a forgery or an observable keystream difference through the Fable data phase has probability at most 2^{-141} ; through Fable-f’s P_6 (segments 2 + 2 + 2: 42 + 33 + 33) at most 2^{-108} . These are trail bounds: the probability of the corresponding *differential*—the sum over all trails with the same input and output difference—is not bounded by them, and the margin over the 2^{-128} target is 13 bits. The two two-round bounds that determine the result (42 and 33) are SAT lower bounds whose searches were stopped, not completed (the last proven levels took 12 328 s and 20 729 s respectively); each additional bit on both would raise the P_8 bound by 4 bits.

4.3 Rotation-constant screening

Model. Candidates $(r_0, r_1, r_2, r_3) \in \{0, \dots, 31\}^4$ were ranked by the minimum trail weight of increasingly large pieces of Fable-P (`sat/rot_screen_sat.py`, data `data/diff/screen_*.csv(.gz)`), with a one-round avalanche statistic as tie-breaker.

Table 6: Degeneration of the screening metrics.

metric	coverage	result
single Q , min trail weight	all $32^4 = 1\,048\,576$	weight 0 for every choice (Lemma 2)
one round of Fable-P	254 576 (all prefixes $r_0 \in \{0..7\}$)	weight 2 for all 201 422 choices without a zero rotation; 0 or 1 otherwise
1.5 rounds (round + column step)	2000 of a 3001-candidate list [†]	max. weight 12; 1338 choices (67%) tie at 12, none above
one-round avalanche mean p	the 1338 tied choices	baseline 0.3558, rank 524/1338; best 0.3814
two rounds, thresholds 16/20/24	top 20 by avalanche + baseline	all 21 have weight ≥ 25 ; baseline ≥ 33

[†] baseline, (20, 12, 8, 7), 630 SIMD-friendly choices with $r_0, r_2 \in \{8, 16, 24\}$, and random choices; stopped after 2000 at 49s per candidate.

Result. Table 6: the trail-weight metrics do not separate the candidates (12 appears to be a structural ceiling at 1.5 rounds—the weight-2 MSB trail plus one column step), and the two-round bounds of the twenty fastest-diffusing candidates are all ≥ 25 with the baseline proved further to ≥ 33 only because it was run longer. The first eight of the twenty are listed in Table 7, the full table in Appendix D. With no differential reason to change, (16, 12, 8, 7) was kept.

Table 7: Top of the two-round ranking (order by one-round avalanche mean; all two-round weights ≥ 25).

rank	(r_0, r_1, r_2, r_3)	1.5-round weight	2-round weight	avalanche mean	SIMD-friendly
1	(24, 21, 16, 20)	12	≥ 25	0.3814	yes
2	(24, 21, 16, 23)	12	≥ 25	0.3814	yes
3	(24, 21, 16, 30)	12	≥ 25	0.3813	yes
4	(16, 21, 24, 9)	12	≥ 25	0.3812	yes
5	(16, 21, 24, 24)	12	≥ 25	0.3812	yes
6	(16, 21, 24, 8)	12	≥ 25	0.3812	yes
7	(16, 5, 8, 6)	12	≥ 25	0.3800	yes
8	(16, 21, 25, 22)	12	≥ 25	0.3792	no
–	(16, 12, 8, 7) baseline	12	≥ 33	0.3558	yes

4.4 Differential-linear calibration

Literature. Table 8 lists the published attacks on reduced-round ChaCha with a 256-bit key; “rounds” are ChaCha single rounds (a Fable-P round is two of them). The attack model is the PRF one: the adversary chooses nonce and counter words and sees the full 512-bit output. Single-bit differential-linear distinguishers exist up to 3.5 rounds [BLT20], a 4-round distinguisher with a 3-bit output mask has bias $0.0032 \approx 2^{-8.3}$ [BLT20], 4.5–5-round distinguishers found by automated (MILP) search [BGG⁺23] reach correlation $2^{-32.2}$ with hull techniques [XXTQ24], and the rest of an attack is 2.5–3 rounds of key guessing with probabilistic neutral bits (whose selection and use were refined in a sequence

of works, e.g. [DGSS22, DGSS23, DGM23, Dey24]) or bit puncturing [FGT25b]. No attack on 8 rounds is known.

Table 8: Key-recovery attacks on reduced-round ChaCha (256-bit key). Time and data as reported.

rounds	time	data	reference
6	2^{139}	2^{30}	[AFK ⁺ 08]
6	$2^{77.4}$	2^{58}	[BLT20, BBC ⁺ 22]
6	$2^{71.0}$	$2^{41.6}$	[Dey24]
6	$2^{57.4}$	$2^{55.7}$	[FGT25b]
7	$2^{248} / 2^{237.7} / 2^{230.86}$	—	[AFK ⁺ 08] / [CM16] / [BLT20]
7	$2^{221.95} / 2^{218.92}$	$2^{90.2} / 2^{87.2}$	[DGSS22] / [DGSS23]
7	$2^{210.3}$	$2^{103.3}$	[WLHL23]
7	$2^{192.89}$	$2^{93.79}$	[Dey24]
7	$2^{189.7}$	$2^{102.63}$	[XXTQ24]
7	$2^{178.12}$	$2^{101.09}$	[SDSM25]
7	$2^{148.2}$	$2^{127.7}$	[FGT25b]
7.25	$2^{223.9} / 2^{212.43}$	$\approx 2^{100.6}$	[XXTQ24] / [SDSM25]
7.5	$2^{255.24}$	$2^{32.64}$	[Dey24]
7.5	$2^{253.23}$	$2^{34.47}$	[SDSM25]
7.5	$2^{250.2}$	$2^{127.1}$	[FGT25b]
7.5 (distinguisher)	$2^{251.4}$	—	[FGT25a]

Experiment. To check that the round constants do not change the differential-linear behaviour, we measured all single-bit differential-linear biases

$$\varepsilon(i, j) = \Pr[\text{output bit } j \text{ unchanged when input bit } i \text{ is flipped}] - \frac{1}{2}$$

over 2^{22} random states per input bit, for 1–4 single rounds (constants injected before each column step, using the same suffix of RC as the corresponding P_n), in three variants: **const** (Fable-P as specified), **noconst** (all constants zero, i.e. the bare ChaCha round structure on a random state) and **chacha** (genuine ChaCha state layout: words 0–3 fixed to the σ constants, key/counter/nonce random, differences only in words 12–15, the published attack model); code `c/dl_bias.c`, data `data/dl/`. The noise level is $\sigma = 2^{-13}$ and the expected extreme over 2^{18} pairs is $4.9\sigma \approx 2^{-9.7}$.

Table 9: Maximum single-bit differential-linear bias $|\varepsilon|$; “rate→rate” restricts both bits to $S[0..7]$.

single r.	Fable r.	const	const, rate→rate	noconst	chacha
1	0.5	0.5 (deterministic)	0.5	0.5	—
2	1	0.5 (deterministic)	0.5	0.5	—
3	1.5	$0.1765 = 2^{-2.50}$	$2^{-4.9}$	$2^{-2.50}$	$0.0991 = 2^{-3.34}$
4	2	$0.00111 = 2^{-9.8}$ (noise ceiling)	$2^{-9.9}$	$2^{-9.8}$	$2^{-9.8}$

Result. Table 9. The pairwise correlation coefficient between the **const** and **noconst** bias maps is 1.0000 at one and two single rounds (constants only shift values, differences are unchanged), 0.9942 at three (the second constant injection changes the carry distribution of some additions; 2473 of 2^{18} pairs move by more than 4σ , but the maximum bias is 0.1765 against 0.1762 and the histogram is unchanged), and -0.003 at four, where both maps are pure noise. Single-bit biases vanish below the $2^{-9.7}$ detection threshold at four single rounds, consistent with the literature’s 3.5-round limit for single-bit distinguishers. The constants leave the bias envelope untouched, so ChaCha results apply to Fable-P round for round.

Margins. With key recovery on 7.5 single rounds as the yardstick, the **Fable** data phase (P_8 , 16 single rounds) has a margin of $2.1\times$, initialisation and finalisation (P_{12} , 24) $3.2\times$, and **Fable-f**'s data phase (P_6 , 12) $1.6\times$; **ChaCha20** itself has $2.7\times$ and **ChaCha12** $1.6\times$. The models differ in a way that we believe favours the defender: a **ChaCha** attacker chooses 128 bits of nonce/counter and sees all 512 output bits (after the feed-forward), while a **Fable** data-phase attacker controls a 256-bit rate difference—more—but observes only the 256-bit rate, and successive data-phase permutations are separated by a 256-bit secret capacity, so the backward (key-guessing) phase of a PNB attack must guess as much as it would guess of a **ChaCha** key. The initialisation permutation, with a known constant prefix, a secret key and a chosen nonce, is the closest analogue of the **ChaCha** PRF and has the largest margin.

4.5 Linear trails and hulls

Model. A linear approximation ($u, v \rightarrow w$) of $z = x + y$ is modelled exactly through the carry chain (Wallén [Wal03], Schulte-Geers [SG13]): with a mask g_i on every carry bit, the local approximation at bit i has correlation 1 if $g_{i+1} = 0$ and $u_i \oplus w_i = v_i \oplus w_i = g_i \oplus w_i = 0$, correlation $\pm \frac{1}{2}$ if $g_{i+1} = 1$ and $(u_i \oplus w_i) \oplus (v_i \oplus w_i) \oplus (g_i \oplus w_i) = 1$, and 0 otherwise (g_0 absent, $g_{32} = 0$). Since each state has at most one successor, the carry-mask path is unique and $|\text{corr}| = 2^{-\#\{i:g_i=1\}}$ exactly; XOR duplicates the output mask onto both inputs, a value used twice receives the XOR of the two masks, and round constants only flip signs (Appendix C). The weight W of a trail gives correlation 2^{-W} (bias 2^{-W-1}).

Results. The quarter-round has a correlation-one linear relation, the LSB relation $z_0 = x_0 \oplus y_0$ carried through four additions (input masks (bit(0), 0, 0, bit(0)); measured correlation +1.0000). One round of **Fable-P** has an exact optimum of weight 1 (input mask $S[0].b_0 \oplus S[4].b_0$, one half-correlation at bit 1 of one addition); its measured correlation is -0.5017 over 2×10^5 samples, the sign coming from the constants. With input and output masks confined to the rate (the keystream-distinguishing model) one round has weight ≥ 28 (SAT lower bound, stopped). Two rounds unconstrained have an exact optimum of weight 20 (correlation 2^{-20}): after “no trail of weight ≤ 19 ” had been proved (18 672 s for that level) a trail of weight 20 was found in 17 s. Its input masks lie in columns 2 and 3, only three bits survive the first round ($S[2].b_0$, $S[7].b_7$, $S[8].b_0$), and the second round spreads them through the LSB relation to 13 words; it belongs to the same LSB family as the one-round optimum, but its intermediate mask is not the one-round optimum's input mask, which is why extending the one-round trail (fixing its input, its output, or placing it in the second round) only produced trails of weight ≥ 48 . Data: `data/lin/`.

Hulls. For a fixed input/output mask pair we enumerate *all* trails up to a weight cap and sum their signed correlations (`sat/lin_hull.py`), computing each sign from the Walsh coefficients of the carry function (a single variable $+\frac{1}{2}$, three variables $-\frac{1}{2}$; at bit 0 the AND with $x \oplus y$ is $-\frac{1}{2}$) and from the constants. For the one-round optimum only one trail exists up to weight 6: no hull effect, and the enumeration reproduces the measured -0.5 including its sign. For two rounds we enumerated two mask pairs (Table 10): around the exact optimum of weight 20, where 15 trails of weight ≤ 24 exist, all with the same sign, and the hull converges to $2^{-18.38}$ (a factor $3.1 \approx 2^{1.6}$) by a cap of 28; and around a weight-48 trail obtained by fixing the input mask to the one-round optimum, where trails of both signs are numerous and the hull converges to $2^{-44.78}$, a factor $9.3 \approx 2^{3.2}$, by a cap of 58.

Result. The best two-round linear correlation is 2^{-20} for a single trail and about $2^{-18.4}$ for its hull; across the two pairs measured, the hull effect at two rounds is a factor 3 to 9,

Table 10: Two-round linear hulls (data/lin/hull_r2opt_w*.json, hull_r2_w*.json).

mask pair	weight cap	trails	positive / negative	hull	vs. best trail
optimum (2^{-20})	24	15	15 / 0	$2^{-18.61}$	$\times 2.6$
	26	35	35 / 0	$2^{-18.45}$	$\times 2.9$
	28	70	70 / 0	$2^{-18.38}$	$\times 3.1$ (converged)
weight-48 trail	54	544	181 / 363	$2^{-45.02}$	$\times 7.9$
	58	7812	3470 / 4342	$2^{-44.78}$	$\times 9.3$
	60	23745	11374 / 12371	$2^{-44.78}$	$\times 9.3$ (converged)

so single-trail bounds for two or more rounds should be read with an allowance of about 3 bits. A heuristic split of P_8 into four two-round segments, each with the exact optimum 20 and a 3-bit allowance, suggests a correlation of about 2^{-68} (2^{136} data for a distinguisher); this is an estimate, not a bound. The one-round LSB relation is isolated and does not extend.

4.6 Rotational-XOR

Model. A rotational pair $(x, x \lll k)$ survives a modular addition with probability $p_k = (1 + 2^{k-32} + 2^{-k} + 2^{-32})/4$ [KN10]: $2^{-1.415}$ for $k = 1$, $2^{-2.0}$ for $k = 16$. This is paid at every one of the 32 additions per round regardless of constants: $2^{-45.3}$ per round, $2^{-271.7}$ for P_6 and $2^{-543.4}$ for P_{12} at $k = 1$. A constant c XORed into a word changes the rotational-XOR difference by $c \oplus (c \lll k)$ [AL16], which must then pass the following additions at a cost of roughly its Hamming weight minus at most 2.

Result. For $k = 1$ the injected weights per round are $\text{hw}(\text{RC}[i] \oplus (\text{RC}[i] \lll 1)) = 14, 18, 10, 16, 16, 18, 16, 10, 16, 14, 16, 14$ for $i = 0..11$, and the same values for the $S[5]$ injection; no round injects a zero difference for any $k \in \{1, \dots, 31\}$ (minimum $\text{hw}(d_0) + \text{hw}(d_5) = 16$, at $k = 4$, round 8). Adding the crude lower bound $\max(0, \text{hw} - 2)$ per injection gives ≥ 148 extra bits for P_6 and ≥ 308 for P_{12} , i.e. rotational-XOR trail probabilities of at most $2^{-419.7}$ for P_6 and $2^{-851.4}$ for P_{12} at $k = 1$ (lower still for other k). Empirically, one quarter-round is rotational with probability $2^{-7.18}$ over 2^{20} samples without a constant (the independent-addition model predicts $2^{-5.66}$; chained additions are known to make the true value smaller [KNP⁺15]), and 0 of 2^{20} after XORing $\text{RC}[6]$ into a . The structure alone puts rotational attacks far below 2^{-256} for six rounds; the constants add to that, and an exact SAT model of rotational-XOR trails with constants was therefore not built. Data: data/rx_check.txt.

4.7 Constants, symmetry, slide and fixed points

The claims of §3.2 were checked by tools/check_constants.py (output data/constants_check.txt):

- the twelve constants recomputed from $\ln p_i$ with 60-digit precision match the table; they are pairwise distinct, none equals another rotated by any amount, no $\text{RC}[i]$ equals $\text{RC}[j] \lll 16$, the minimum pairwise Hamming distance is 9, and $\text{RC}[i] \oplus (\text{RC}[i] \lll 16) \neq 0$ for all i ;
- with constants set to zero, the all-zero state is a fixed point of 12 rounds and the column-symmetric subspace $\mathcal{V}$ is preserved by 1–12 rounds, whereas the row-symmetric and all-equal classes are left after one round;

3. with the constants, every structured input (all-zero, all-one, all-MSB, all-1, three random members each of $\mathcal{V}$, of the row-symmetric class and of the all-equal class) gives an output after $n = 1..12$ rounds—counted both as a prefix and as a suffix of P_{12} —that is neither symmetric nor a fixed point, and 10^4 random members of $\mathcal{V}$ leave $\mathcal{V}$ after every one of the twelve rounds;
4. slide: no two constants are equal, no constant is a rotation of another, the suffix $\text{RC}[6..11]$ used by P_6 is not periodic and does not occur as another window of the table, and the eleven consecutive XOR differences $\text{RC}[i] \oplus \text{RC}[i + 1]$ are distinct (no affine slide);
5. fixed points: none of the structured inputs is a fixed point of any P_n , and 2^{18} random states contain no one-round fixed point. A random permutation has about one fixed point; only structured ones can be excluded.

4.8 Key commitment

Let $X = (Y_R, Y_C \oplus K)$ be the state just before the finalising permutation, where (Y_R, Y_C) is the state after the last (padded) message block has been XORed into the rate; $T = \text{trunc}_\tau(P_{12}(X)[0..7])$. X is a deterministic function of (K, N, A, M) and of the parameter set (through IV_1), and for a fixed (C, T) and (K', N', A') the message M' is determined by decryption, so CMT-1 (two keys) and CMT-4 (two distinct tuples) in the sense of Bellare and Hoang [BH22] both amount to producing two tuples with the same (C, T) .

Theorem 1 (key commitment). *Model P_{12} as a random permutation π of $\{0, 1\}^{512}$ to which the adversary makes q forward or inverse queries. Any adversary that outputs two distinct tuples $(K, N, A, M) \neq (K', N', A', M')$ encrypting to the same (C, T) under Fable-AEAD with a τ -bit tag succeeds with probability at most*

$$\frac{q(q-1)}{2^{\tau+1}} + \frac{q(q-1)}{2^{513}} + \frac{q}{2^{256}}.$$

Proof sketch. Let X, X' be the two pre-finalisation states. (i) If $X \neq X'$, then the τ -bit truncations of $\pi(X)$ and $\pi(X')$ coincide: a collision of the τ -bit truncation of π on two distinct queried inputs, probability at most $\binom{q}{2}/2^\tau$ up to the negligible bias of a truncated permutation. (ii) If $X = X'$ and $K \neq K'$, then $Y_R = Y'_R$ and $Y_C \oplus Y'_C = K \oplus K'$: two duplex paths through π , ending in different queries, whose final states satisfy a fixed 512-bit relation, probability at most $\binom{q}{2}/2^{512}$. (If $K = K'$ and the tuples differ elsewhere, $X = X'$ is a full internal state collision, covered by the same count.) (iii) Otherwise the adversary has guessed the 256-bit capacity of some path, probability at most $q/2^{256}$. A successful output is witnessed by one of the three events, and values of π that were never queried are uniform and cannot produce an equal tag except by the same counting. $\square$

Corollary 1. *With $\tau = 256$ the bound is $\approx q^2/2^{257}$: about 2^{128} permutation evaluations. With $\tau = 128$ it is $\approx q^2/2^{129}$: about 2^{64} . Both are tight, since a generic birthday search on $\text{trunc}_\tau \circ P_{12}$ achieves them. Commitment is an offline collision problem and is not affected by the number of users or by nonce reuse. The specification’s claims “128-bit commitment with 256-bit tags, 64-bit with 128-bit tags” are therefore correct in the ideal-permutation model.*

The model’s practical premise is that P_{12} admits no collision shortcut, i.e. no 12-round differential with zero rate-output difference of probability above about 2^{-128} (the cost of the generic birthday attack); the split of the two-round bound gives trail probabilities $\leq 2^{-198}$ for twelve rounds (6×33), and 24 single rounds are beyond every published ChaCha differential or collision-type result (§4.4). The design choice behind the theorem is discussed in §3.6; the argument is written out in `docs/key-commitment.md`.

4.9 Multi-user and STREAM bounds

Model. Fable-AEAD is a keyed duplex with $b = 512$, $r = c = 256$, $k = 256$; μ users hold independent keys; the adversary makes N offline permutation evaluations, M online data blocks in total (32 bytes each, plus two calls per message for initialisation and finalisation) and q_D verification queries; nonces are respected. Writing the multi-user keyed-duplex bound of Daemen, Mennink and Van Assche [DMV17] (which refines Jovanovic–Luykx–Mennink [JLM14]) with its multiplicity terms L, Ω collapsed to their worst case $\leq M$ and constants omitted,

$$\text{Adv} \leq \underbrace{\frac{\mu N}{2^k}}_{[A]} + \underbrace{\frac{MN}{2^c}}_{[B]} + \underbrace{\frac{M^2}{2^{c+1}}}_{[C]} + \underbrace{\frac{(M+N)^2}{2^{b+1}}}_{[D]} + \underbrace{\frac{q_D}{2^\tau}}_{[E]} + \underbrace{\frac{F^2}{2^{153}}}_{[F]},$$

where $[F]$ is the probability that two of F random 152-bit file nonces under one key coincide (zero for structured N_f). Table 11 evaluates the terms at the specification’s limits.

Table 11: Terms of the multi-user bound at $\mu = 2^{32}$ users, 2^{64} bytes per user ($M \approx 2^{91}$), $N = 2^{128}$, $q_D = 2^{64}$.

term	governed by	value	single user ($M = 2^{59}$)
$[A] \mu N / 2^k$	$k = 256$	2^{-96}	2^{-128}
$[B] MN / 2^c$	$c = 256$	2^{-37}	2^{-69}
$[C] M^2 / 2^{c+1}$	$c = 256$	2^{-75}	2^{-139}
$[D] (M+N)^2 / 2^{b+1}$	$b = 512$	2^{-256}	2^{-256}
$[E] q_D / 2^\tau$	τ	2^{-192} ($\tau=256$), 2^{-64} ($\tau=128$)	same
$[F] F^2 / 2^{153}$	$ N_f = 152$	2^{-89} ($F=2^{32}$) to 2^{-57} ($F=2^{48}$)	same

STREAM. Hoang et al. [HRRV15] prove that the nOAE advantage of STREAM over an AEAD scheme is at most the nAE advantage of that scheme against an adversary with the same number of *chunks* as queries, provided the derived chunk nonces are distinct; with $N_i = N_f \parallel i \parallel \text{flag}_i$ they are distinct within a file by construction and across files whenever N_f is. The reduction is lossless, so Fable-Stream inherits the bound above with M counted in chunks times 2051 permutation calls per 64 KiB chunk (2048 data blocks, one associated-data block, initialisation and finalisation), a factor of 1.0015 on the block count.

Result. (1) $c = 256$ puts the keyed duplex in the “beyond $2^{c/2}$ ” regime: with 2^{64} bytes and 2^{128} offline work a single user retains 2^{-69} of advantage, and $[B]$ would exceed 1 in the multi-user setting at $c = 192$ (§3.5). (2) Multi-user degradation is governed by $[A]$ and $[B]$, both linear in μ — $\log_2 \mu$ bits, as the specification claims; $[C]$ grows as μ^2 but from 2^{-139} . (3) The largest concrete term is the random-file-nonce collision $[F]$, 2^{-57} to 2^{-89} , which is why the specification limits random N_f to 2^{40} files per key (2^{-73}) and offers a structured alternative. The derivation with the DMV17 multiplicity terms is in docs/security-bounds.md.

4.10 Fable-Stream nonce

Within one file the chunk nonces $N_f \parallel \text{LE}_{32}(i) \parallel \text{flag}_i$ are distinct by construction for up to 2^{32} chunks. Across files under one key the 152-bit N_f must be unique; when it is drawn at random this holds except with probability $F^2 / 2^{153}$ ($[F]$ above), and when it is structured (for instance a 64-bit device or session identifier followed by an 88-bit counter maintained

by the caller) it holds unconditionally. Nonce reuse is not tolerated: as for any nonce-based scheme without misuse resistance, a repeated (K, N_i) pair reveals the XOR of the two chunk plaintexts and the keystream of that position, and lets the adversary forge for that nonce.

4.11 What is not covered

The following are not established by this paper and are listed so that they are not mistaken for results:

- direct (non-split) differential trail bounds for three or more rounds; the three-round rate-restricted bound of ≥ 43 is the furthest the solver reached within its time cap;
- exact differential optima for two rounds (≥ 33 unconstrained, ≥ 42 rate-restricted); those searches were stopped, not completed (the two-round linear optimum, by contrast, is exact);
- differential (clustering) probabilities as opposed to trail probabilities; the 13-bit margin of the 2^{-141} split bound over 2^{-128} does not cover clustering, and the corresponding hull allowance for linear cryptanalysis was measured only for one two-round mask pair;
- algebraic, cube and integral attacks, and guess-and-determine attacks on the initialisation;
- fault, probing and other physical attacks (excluded by the specification);
- dedicated analysis of Fable-Hash beyond the flat sponge claim inherited from the permutation;
- any third-party cryptanalysis.

5 Implementation and Performance

5.1 Implementations

Three implementations exist in the repository. The Python reference (`fable.py`) is the normative one and is deliberately slow and literal. The portable C implementation (`c/fable_p.c`, `c/fable_aead.c`) is scalar and word-oriented, with the rotation amounts as compile-time constants so that the compiler emits immediate rotates. The AVX2 implementation (`c/fable_p_avx2.c`, `c/fable_stream_avx2.c`) processes eight Fable-Stream chunks at once: each of the sixteen `ymm` registers holds the same state word of eight independent states (lane = chunk), so the quarter-round is executed with `vpaddq/vpxor`, the rotations by 16 and 8 as byte shuffles (`vpshufb`) and those by 12 and 7 as shift-or pairs. The eight states stay in this transposed layout for the whole AEAD—initialisation (P_{12}), the header block, every data block (P_d) and finalisation (P_{12})—and data enters and leaves through an 8×8 transpose of 32-bit words (three `unpack/permute2x128` stages), twice per 32-byte block. Chunk groups of fewer than eight and the final chunk take the scalar path. Decryption mirrors encryption: the transposed ciphertext block is XORed with the rate to give the plaintext and then *replaces* the rate; the last partial block is recovered from the keystream and its padded plaintext XORed back; the eight tags are recomputed and compared lane by lane in constant time, and a failure in any lane zeroes the whole output and returns an error.

Cross-validation covers, for both parameter sets: the permutation vectors $P_{12}(0)$ and $P_8(0)$; the ten AEAD and two hash vectors of `test_vectors_v0.3.json` (the Fable-f

vectors equal the v0.1 vectors); 50 random states for every round count 1–12 and 200 random rotation-constant sets against the Python permutation; 36 random (message length, AD length) combinations per set against Python encryption and C decryption; nine chunk-size/length combinations of **Fable-Stream** per set (chunk sizes 32, 1000, 4096, 65536 bytes; zero to three AVX2 groups; empty and partial final chunks; chunk sizes that are not multiples of 32) with AVX2 and scalar encryption equal to Python, AVX2 and scalar decryption returning the plaintext, Python decrypting the C output, and single-bit tampering in a middle chunk, tampering of the final tag, truncation and decryption under the wrong parameter set all rejected; and the AVX2 permutation against the scalar one on 2000 random states $\times$ 8 lanes for every round count 1–12 (`tests/check_c_vectors.py`, `tests/check_stream_avx2.py`, `c/test_avx2.c`).

5.2 Constant-time properties

By construction the implementations contain no table look-ups and no branches on secret data: the permutation is straight-line ARX code, block handling depends only on lengths, the tag comparison accumulates the OR of byte differences and tests it once, and the AVX2 lanes do not interact except through the data-independent transposes. We state this as a property of the code as written, not as a measured result: no *dudect*- or *ctgrind*-style verification, and no measurement on a real target, has been carried out (§5.5).

5.3 Measurement method

All measurements were taken on one machine: an Intel Raptor Lake desktop processor (8 performance cores with hyper-threading and 8 efficiency cores, 24 logical CPUs) under Windows 11, code compiled with clang 21.1.0 at `-O3 -mavx2`. Cycle counts use `rdtsc`; because the TSC runs at a fixed reference frequency, the actual core clock is calibrated in the same run by timing a chain of 4×10^8 dependent one-cycle additions, and cycles per byte (cpb) are reported in core cycles. The benchmark process runs at high priority with its main thread pinned to logical CPU 0 (a performance core) and, for multi-threaded runs, each worker pinned to a distinct performance core; every figure is the best of three (permutation micro-benchmarks: five) repetitions. Two caveats apply. First, background SAT searches were running at low priority during all measurements, and the calibrated core clock varied between runs (about 3.0 GHz under heavy load for the permutation micro-benchmarks, 3.65 GHz for the first **Fable-Stream** measurement, 4.89 GHz for the final one); absolute GB/s figures therefore differ between runs while cpb figures agree to within a few percent, and cpb is the number to compare. Second, an early run that left the main thread unpinned was scheduled on an efficiency core and reported roughly half the throughput; all figures below are from pinned runs.

5.4 Results

Permutation. Table 12 gives the cost of the permutation alone (heavy-load run, core clock $\approx$ 3.0 GHz). The scalar P_6 costs about 37 cycles per round, in line with a **ChaCha** double round; the eight-lane core is $4.4\times$ faster per byte than the scalar code, and the transposes cost about 200 cycles per call, a third of a P_6 call, which is why **Fable-Stream** keeps the states transposed across the whole AEAD.

Fable-Stream. Table 13 gives end-to-end throughput for 1 GiB of input in 64 KiB chunks (final run, core clock 4.89 GHz). Single-threaded, the default parameter set encrypts at 2.00 GB/s (2.44 cpb) and decrypts at the same speed; **Fable-f** reaches 2.53 GB/s (1.93 cpb). The earlier **Fable-f** measurement at 3.65 GHz gave 1.97 GB/s and 1.85 cpb, consistent in cpb. P_8 costs 21 % of throughput relative to P_6 although it has 33 % more permutation

Table 12: Permutation cost, single thread (`data/bench_result.txt`). cpb = cycles per 32-byte data block.

variant	core cycles/call	cpb	ns/call
scalar P_6	222	6.9	74
scalar P_{12}	424	13.3	141
AVX2 $\times 8$ P_6 , incl. transposes	607 (8 states)	2.4	202
AVX2 $\times 8$ P_{12} , incl. transposes	992	3.9	330
AVX2 $\times 8$ P_6 , core only	404	1.6	134
AVX2 $\times 8$ P_{12} , core only	805	3.1	268

749 rounds, because the two P_{12} calls per chunk, the header block and the transposes are the
 750 same for both. With four and eight threads the scaling is 3.1–3.3 $\times$ and 5.0–5.7 $\times$; the
 751 sub-linearity comes from the lower all-core turbo frequency and from memory bandwidth
 752 (at 11.5 GB/s in and out the traffic is a substantial fraction of what the memory system
 753 delivers), not from the algorithm, since chunks are independent.

Table 13: Fable-Stream throughput, 1 GiB, 64 KiB chunks, best of three (`data/bench_stream_v03.txt`). cpb is given for single-thread runs only.

set	path	threads	GB/s	cpb	speed-up
Fable (P_8)	scalar, encrypt	1	0.48	10.2	–
Fable	scalar, decrypt	1	0.51	9.6	–
Fable	AVX2 $\times 8$, encrypt	1	2.00	2.44	1 $\times$
Fable	AVX2 $\times 8$, decrypt	1	2.05	2.39	–
Fable	AVX2 $\times 8$, encrypt	4	6.54	–	3.3 $\times$
Fable	AVX2 $\times 8$, decrypt	4	6.31	–	3.1 $\times$
Fable	AVX2 $\times 8$, encrypt	8	11.5	–	5.7 $\times$
Fable	AVX2 $\times 8$, decrypt	8	10.2	–	5.0 $\times$
Fable-f (P_6)	scalar, encrypt	1	0.62	7.9	–
Fable-f	AVX2 $\times 8$, encrypt	1	2.53	1.93	1 $\times$
Fable-f	AVX2 $\times 8$, decrypt	1	2.59	1.89	–
Fable-f	AVX2 $\times 8$, encrypt	4	8.24	–	3.3 $\times$
Fable-f	AVX2 $\times 8$, encrypt	8	14.5	–	5.7 $\times$

754 **Comparison.** Table 14 shows ChaCha20-Poly1305 and AES-256-GCM on the same
 755 machine, measured in the same session as Table 13 with the process pinned to logical CPU
 756 0 at high priority (via `psutil`), i.e. under the same conditions as the Fable runs. The
 757 OpenSSL 3.5 figures were obtained through the `cryptography` Python package (whose
 758 wheel bundles OpenSSL) on a 1 GiB buffer, once as a single call and once as a loop over
 759 64 KiB pieces (the loop is faster, presumably from cache effects), and through `openssl`
 760 `speed` on 16 KiB blocks. The libsodium figure comes from the `pynacl` wheel, whose bundled
 761 build evidently lacks the AVX2 path, and is listed only to explain why it is not used as a
 762 reference. The default Fable parameter set is on par with OpenSSL’s ChaCha20-Poly1305
 763 (2.00 against 1.57–2.37 GB/s depending on the measurement), and Fable-f is 7–60 % faster;
 764 both include authentication. AES-256-GCM with AES-NI and PCLMULQDQ is three times
 765 faster than either. This places Fable in the niche of ChaCha20: platforms without AES
 766 acceleration, constant time at no cost, and—beyond ChaCha20-Poly1305—a random-safe
 767 nonce and key commitment; where AES-NI is available and those properties are not needed,
 768 AES-GCM and AEGIS [WP14] are faster.

Table 14: Reference AEADs on the same machine, single thread pinned to logical CPU 0 at high priority (`data/bench_chacha_v04_pinned.txt`, `bench_openssl_speed_v04_pinned.txt`).

implementation	GB/s
ChaCha20-Poly1305, OpenSSL 3.5 (cryptography 48), one 1 GiB call	1.57
ChaCha20-Poly1305, OpenSSL 3.5, 64 KiB pieces	1.94
ChaCha20-Poly1305, OpenSSL 3.5.4 speed -evp, 16 KiB blocks	2.37
AES-256-GCM, OpenSSL 3.5.4 speed -evp, 16 KiB blocks (AES-NI)	6.07
ChaCha20-Poly1305, libsodium (pynacl 1.6.2 build, no AVX2 path)	0.75
Fable, AVX2 $\times 8$ (Table 13)	2.00
Fable-f, AVX2 $\times 8$	2.53

5.5 Not measured

No ARM/NEON implementation exists (the transposed eight-lane design maps to four lanes of 128-bit registers, but no measurement was made); x86 without AVX2 was not measured beyond the scalar path; latency of a single short Fable-AEAD message (below one 64 KiB chunk, where the two P_{12} calls dominate) was not measured; and the constant-time properties of §5.2 were not verified empirically.

6 Conclusion and Open Problems

Fable puts one new object on the table—a 512-bit ARX permutation that is the ChaCha double round with round constants—and builds everything else from constructions with existing proofs: a keyed duplex whose tag is the truncated output of the finalising permutation and therefore commits to the key, the nonce, the associated data and the message at no extra cost; a STREAM mode whose chunks are independent enough to fill eight AVX2 lanes and to be decrypted online; and a sponge hash. The first-party analysis gives exact one-round trail optima and an exact two-round linear optimum, SAT lower bounds for the two-round differential ones, a split bound of 2^{-141} on any forgery trail through the default data phase, a differential-linear calibration showing that the constants do not alter the ChaCha bias envelope (so that the $2.1\times$ and $3.2\times$ round-count margins can be read against the ChaCha literature), a hull enumeration, rotational-XOR, symmetry and slide checks, and mode-level bounds; the AVX2 implementation runs at 2.00 GB/s single-threaded, on par with OpenSSL’s ChaCha20-Poly1305 on the same machine, and 2.53 GB/s for Fable-f.

Open problems , in the order in which we think they matter:

1. *Two-round differential optima and clustering.* The 2^{-141} bound rests on two SAT lower bounds (42 rate-restricted, 33 unconstrained) that were stopped, not completed, and it bounds trails, not differentials. Finding the exact two-round differential optima (the linear one, weight 20, took 5.2 CPU-hours for its last level and is done), and estimating how many trails share the input/output differences of the best ones, would say whether the 13-bit margin over 2^{-128} is real. Every bit gained on both two-round bounds adds four bits to the P_8 bound.
2. *A dedicated attack on the data phase.* We calibrated Fable-P against ChaCha but did not attempt a differential-linear or PNB-style attack in the duplex model itself, where the adversary controls a 256-bit rate difference, sees only 256 output bits and faces a 256-bit secret capacity. Whether that model is easier or harder than the ChaCha PRF model for 16 single rounds is the question the round count depends on.

- 803 3. *Algebraic and cube attacks on initialisation*. The initialisation permutation has a
804 64-bit constant prefix, a 256-bit key and a 192-bit chosen nonce; cube and conditional-
805 cube attacks on 24 single rounds have not been evaluated.
- 806 4. *Fable-Hash*. Only the flat sponge claim inherited from the permutation is made;
807 collision and preimage attacks on reduced rounds of the hash have not been studied.
- 808 5. *NEON and short messages*. No ARM implementation exists, and the latency of a
809 single message shorter than a chunk, where the two P_{12} calls dominate, has not been
810 measured.
- 811 6. *Side channels*. The constant-time properties are argued from the code and have not
812 been tested on hardware.

813 **Status.** All analysis reported here is first-party; no third party has examined *Fable*, and
814 the differential bounds beyond two rounds are split bounds rather than direct ones. *Fable*
815 must not be used to protect real data until independent cryptanalysis has been published.
816 The specification, the reference and C/AVX2 implementations, every script and every raw
817 data file behind the numbers in this paper, and the analysis log that records what was
818 tried and what failed, are available at <repository URL>; a preprint is at <ePrint URL>.
819 We invite analysis of the permutation, of the rate-restricted duplex model, and of the
820 claims in §4.

821 References

- 822 [AFK⁺08] Jean-Philippe Aumasson, Simon Fischer, Shahram Khazaei, Willi Meier, and
823 Christian Rechberger. New features of Latin dances: Analysis of Salsa, ChaCha,
824 and Rumba. In *FSE 2008*, volume 5086 of *LNCS*, pages 470–488. Springer,
825 2008.
- 826 [AJN14] Jean-Philippe Aumasson, Philipp Jovanovic, and Samuel Neves. NORX: Parallel
827 and scalable AEAD. In *ESORICS 2014, Part II*, volume 8713 of *LNCS*, pages
828 19–36. Springer, 2014.
- 829 [AL16] Tomer Ashur and Yunwen Liu. Rotational cryptanalysis in the presence of
830 constants. *IACR Transactions on Symmetric Cryptology*, 2016(1):57–70, 2016.
- 831 [Arc20] Scott Arciszewski. XChaCha: eXtended-nonce ChaCha and
832 AEAD_XChaCha20_Poly1305. Internet-Draft draft-irtf-cfrg-xchacha-
833 03, 2020.
- 834 [Aum20] Jean-Philippe Aumasson. Too much crypto. Cryptology ePrint Archive, Paper
835 2019/1492, 2020. <https://eprint.iacr.org/2019/1492>.
- 836 [BBC⁺22] Christof Beierle, Marek Broll, Federico Canale, Nicolas David, Antonio Flórez-
837 Gutiérrez, Gregor Leander, María Naya-Plasencia, and Yosuke Todo. Improved
838 differential-linear attacks with applications to ARX ciphers. *Journal of Cryptology*,
839 35(4):29, 2022.
- 840 [BDPV12] Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche. Duplex-
841 ing the sponge: Single-pass authenticated encryption and other applications.
842 In *SAC 2011*, volume 7118 of *LNCS*, pages 320–337. Springer, 2012.
- 843 [Ber08a] Daniel J. Bernstein. ChaCha, a variant of Salsa20. In *Workshop Record of*
844 *SASC 2008: The State of the Art of Stream Ciphers*, 2008. [https://cr.yp.
845 to/chacha/chacha-20080128.pdf](https://cr.yp.to/chacha/chacha-20080128.pdf).

- [Ber08b] Daniel J. Bernstein. The Salsa20 family of stream ciphers. In *New Stream Cipher Designs: The eSTREAM Finalists*, volume 4986 of *LNCS*, pages 84–97. Springer, 2008.
- [BFFH20] Armin Biere, Katalin Fazekas, Mathias Fleury, and Maximillian Heisinger. CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT competition 2020. Technical report, SAT Competition 2020 – Solver and Benchmark Descriptions, 2020.
- [BGG⁺23] Emanuele Bellini, David G rault, Juan Grados, Rusydi H. Makarim, and Thomas Peyrin. Fully automated differential-linear attacks against ARX ciphers. In *CT-RSA 2023*, volume 13871 of *LNCS*, pages 252–276. Springer, 2023. ePrint 2023/181.
- [BH22] Mihir Bellare and Viet Tung Hoang. Efficient schemes for committing authenticated encryption. In *EUROCRYPT 2022, Part II*, volume 13276 of *LNCS*, pages 845–875. Springer, 2022.
- [BLT20] Christof Beierle, Gregor Leander, and Yosuke Todo. Improved differential-linear attacks with applications to ARX ciphers. In *CRYPTO 2020, Part III*, volume 12172 of *LNCS*, pages 329–358. Springer, 2020.
- [CB18] Paul Crowley and Eric Biggers. Adiantum: Length-preserving encryption for entry-level processors. *IACR Transactions on Symmetric Cryptology*, 2018(4):39–61, 2018.
- [CM16] Arka Rai Choudhuri and Subhamoy Maitra. Significantly improved multi-bit differentials for reduced round Salsa and ChaCha. *IACR Transactions on Symmetric Cryptology*, 2016(2):261–287, 2016.
- [CR22] John Chan and Phillip Rogaway. On committing authenticated-encryption. In *ESORICS 2022, Part II*, volume 13555 of *LNCS*, pages 275–294. Springer, 2022.
- [CS21] Murilo Coutinho and Tertuliano C. Souza Neto. Improved linear approximations to ARX ciphers and attacks against ChaCha. In *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 711–740. Springer, 2021.
- [DEMS21] Christoph Dobraunig, Maria Eichlseder, Florian Mendel, and Martin Schl  ffer. Ascon v1.2: Lightweight authenticated encryption and hashing. *Journal of Cryptology*, 34(3):33, 2021.
- [Dey24] Sabyasachi Dey. Advancing the idea of probabilistic neutral bits: First key recovery attack on 7.5 round ChaCha. *IEEE Transactions on Information Theory*, 70(8):6091–6106, 2024.
- [DGM23] Sabyasachi Dey, Hirendra Kumar Garai, and Subhamoy Maitra. Cryptanalysis of reduced round ChaCha – new attack & deeper analysis. *IACR Transactions on Symmetric Cryptology*, 2023(1):89–110, 2023.
- [DGSS22] Sabyasachi Dey, Hirendra Kumar Garai, Santanu Sarkar, and Nitin Kumar Sharma. Revamped differential-linear cryptanalysis on reduced round ChaCha. In *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 86–114. Springer, 2022.
- [DGSS23] Sabyasachi Dey, Hirendra Kumar Garai, Santanu Sarkar, and Nitin Kumar Sharma. Enhanced differential-linear attacks on reduced round ChaCha. *IEEE Transactions on Information Theory*, 69(8):5318–5336, 2023.

- [DL25] Frank Denis and Samuel Lucas. The AEGIS family of authenticated encryption algorithms. Internet-Draft draft-irtf-cfrg-aegis-aead, 2025. TODO-verify draft version.
- [DMV17] Joan Daemen, Bart Mennink, and Gilles Van Assche. Full-state keyed duplex with built-in multi-user support. In *ASIACRYPT 2017, Part II*, volume 10625 of *LNCS*, pages 606–637. Springer, 2017.
- [FGT25a] Antonio Flórez-Gutiérrez and Yosuke Todo. Divide-and-conquer trail enumeration puncturing: Application to Salsa and ChaCha. In *ASIACRYPT 2025, Part I*, LNCS. Springer, 2025.
- [FGT25b] Antonio Flórez-Gutiérrez and Yosuke Todo. Improved cryptanalysis of ChaCha: Beating PNBs with bit puncturing. In *EUROCRYPT 2025, Part I*, volume 15601 of *LNCS*. Springer, 2025. ePrint 2025/437.
- [GLL19] Shay Gueron, Adam Langley, and Yehuda Lindell. AES-GCM-SIV: Nonce misuse-resistant authenticated encryption. RFC 8452, 2019.
- [HRRV15] Viet Tung Hoang, Reza Reyhanitabar, Phillip Rogaway, and Damian Vizár. Online authenticated-encryption and its nonce-reuse misuse-resistance. In *CRYPTO 2015, Part I*, volume 9215 of *LNCS*, pages 493–517. Springer, 2015.
- [IMMS18] Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva. PySAT: A Python toolkit for prototyping with SAT oracles. In *SAT 2018*, volume 10929 of *LNCS*, pages 428–437. Springer, 2018.
- [JLM14] Philipp Jovanovic, Atul Luykx, and Bart Mennink. Beyond $2^{c/2}$ security in sponge-based authenticated encryption modes. In *ASIACRYPT 2014, Part I*, volume 8873 of *LNCS*, pages 85–104. Springer, 2014.
- [KN10] Dmitry Khovratovich and Ivica Nikolić. Rotational cryptanalysis of ARX. In *FSE 2010*, volume 6147 of *LNCS*, pages 333–346. Springer, 2010.
- [KNP⁺15] Dmitry Khovratovich, Ivica Nikolić, Josef Pieprzyk, Przemysław Sokołowski, and Ron Steinfeld. Rotational cryptanalysis of ARX revisited. In *FSE 2015*, volume 9054 of *LNCS*, pages 519–536. Springer, 2015.
- [LM02] Helger Lipmaa and Shiho Moriai. Efficient algorithms for computing differential properties of addition. In *FSE 2001*, volume 2355 of *LNCS*, pages 336–350. Springer, 2002.
- [MP13] Nicky Mouha and Bart Preneel. Towards finding optimal differential characteristics for ARX: Application to Salsa20. Cryptology ePrint Archive, Paper 2013/328, 2013. <https://eprint.iacr.org/2013/328>.
- [NL18] Yoav Nir and Adam Langley. ChaCha20 and Poly1305 for IETF protocols. RFC 8439, 2018.
- [SDSM25] Nitin Kumar Sharma, Sabyasachi Dey, Santanu Sarkar, and Subhamoy Maitra. On improved cryptanalytic results against ChaCha for reduced rounds ≥ 7 . Cryptology ePrint Archive, Paper 2025/428, 2025. <https://eprint.iacr.org/2025/428>.
- [SG13] Ernst Schulte-Geers. On CCZ-equivalence of addition mod 2^n . *Designs, Codes and Cryptography*, 66(1–3):111–127, 2013.

- [Wal03] Johan Wallén. Linear approximations of addition modulo 2^n . In *FSE 2003*, volume 2887 of *LNCS*, pages 261–273. Springer, 2003.
- [WLHL23] Shichang Wang, Meicheng Liu, Shiqi Hou, and Dongdai Lin. Moving a step of ChaCha in syncopated rhythm. In *CRYPTO 2023, Part III*, volume 14083 of *LNCS*, pages 273–304. Springer, 2023.
- [WP14] Hongjun Wu and Bart Preneel. AEGIS: A fast authenticated encryption algorithm. In *SAC 2013*, volume 8282 of *LNCS*, pages 185–201. Springer, 2014.
- [XXTQ24] Zhichao Xu, Hong Xu, Lin Tan, and Wenfeng Qi. Differential-linear cryptanalysis of reduced round ChaCha. *IACR Transactions on Symmetric Cryptology*, 2024(2):166–189, 2024.

A Round constants

$RC[i] = \lfloor 2^{32} \cdot \text{frac}(\ln p_i) \rfloor$ for the first twelve primes. Recomputed with 60-digit precision in `tools/check_constants.py`; all twelve are distinct, no two are rotations of each other, and $RC[i] \oplus (RC[i] \lll 16) \neq 0$ for every i .

i	p_i	$RC[i]$	i	p_i	$RC[i]$
0	2	b17217f7	6	17	d54d783f
1	3	193ea7aa	7	19	f1c6c0c0
2	5	9c041f7e	8	23	22afbfbfa
3	7	f2272ae3	9	29	5e071979
4	11	65dc76ef	10	31	6f19c912
5	13	90a08566	11	37	9c651dc7

B Test vectors

All values are byte strings in hexadecimal; state words are serialised little-endian, $S[0]$ first. Key $K = 00\ 01 \dots 1f$ (bytes 0..31), nonce $N = 00\ 01 \dots 17$ (bytes 0..23).

Permutation of the all-zero state.

$P_{12}(0^{512})$	987e1eab32cefb5c1476a1517c5d0c9f81d61cb4808b32b00db6745079ed44947ea9cb25720c64fe7bb9b86e3e1db2d345a9b4024a1f5849b1f77ecc6188a8fd
$P_8(0^{512})$	13bf4c33b2ac964ca319d28c2e01c0732aaf9109228e083ec58861cab663dd6a7bbce1ed69452d5975b4def6f81f40812bfceced5a62100a16f9ef2e066e302

AEAD, one vector per parameter set. $A = \text{"header"}$ (686561646572), $M = \text{bytes 0..31}$ (000102...1f); output is $C \parallel T$ (64 bytes).

Fable (P_{12}/P_8)	59358977baf9ca6b482719c0299011ab2dd25d2c03f933b17a3822ff0cfdd25448f4dfc93df45855b8f92ea1282cafaf5087a4077113e4b98949ddec135aaa37
Fable-f (P_{12}/P_6)	9e43775b205f324f6fd91b0511fa4a401bf73905ea86c0066a2f94f49988c9263d6d58e99c0b8d8adf3dfb8df17ec96ef186b62d0118f8915c652a5b5a1ff768

The remaining vectors (empty message, AD only, 5-byte message, two blocks; hash of the empty string and of "abc") are in `test_vectors_v0.3.json`; `tests/check_c_vectors.py` and `tests/check_stream_avx2.py` verify the C implementation against all of them and against the Python reference on random inputs.

C SAT models

Both models are in `sat/` of the repository (`fable_diff.py`, `fable_lin.py`, `lin_hull.py`) and use PySAT [IMMS18] with CaDiCaL 1.5.3 [BFFH20] (Glucose 4 only where interruptible solving was needed).

Differential model. Every 32-bit difference on a wire is a vector of 32 Boolean variables. XOR $z = x \oplus y$ introduces a fresh word z with four clauses per bit; a rotation is a re-indexing. For $z = x + y$ with differences (α, β, γ) we add, for $i = 0..30$, a weight variable w_i with the clauses $\neg w_i \Rightarrow (\alpha_i = \beta_i = \gamma_i)$, $w_i \Rightarrow \neg(\alpha_i = \beta_i = \gamma_i)$ and $\neg w_i \Rightarrow (\alpha_{i+1} \oplus \beta_{i+1} \oplus \gamma_{i+1} \oplus \alpha_i = 0)$ (14 clauses per bit), plus $\alpha_0 \oplus \beta_0 \oplus \gamma_0 = 0$; this is the Lipmaa–Moriai characterisation [LM02], the trail weight is $\sum w_i$, and the quarter-round is the sequence of four additions, four XORs and four rotations of §2.2. One round is 32 additions (992 weight variables, 3552 variables, 18 112 clauses); two rounds double this. A clause requires the input difference to be nonzero. The rate-restricted variants add unit clauses fixing the capacity part of the input (`rate-in`) or output (`cap-out-zero`) difference to zero, or a clause requiring a nonzero rate output difference. The total weight is bounded by an incremental totalizer (PySAT `ITotalizer`); the search asks “weight $\leq k$?” for $k = k_0, k_0 + 1, \dots$ and stops at the first satisfiable k (exact optimum) or at a time cap (lower bound $k + 1$). Found trails are decoded to hexadecimal differences at every round boundary and verified by sampling random pairs through the C permutation.

Linear model. Masks are again 32 Boolean variables per wire. For an addition with masks (u, v, w) we introduce carry-mask variables $g_1, \dots, g_{31}$ and encode, for $i = 1..31$: $g_{i+1} = 0 \Rightarrow u_i = v_i = g_i = w_i$ and $g_{i+1} = 1 \Rightarrow u_i \oplus v_i \oplus g_i \oplus w_i = 1$ (with $g_{32} = 0$ forcing $u_{31} = v_{31} = g_{31} = w_{31}$), and at bit 0: $g_1 = 0 \Rightarrow u_0 = v_0 = w_0$. The weight is $\sum_{i=1}^{31} g_i$ and the trail correlation $2^{-\text{weight}}$ exactly (§4.5). XOR gates copy the output mask to both inputs; a wire used twice carries the XOR of the two consumers’ masks, which is realised by a fresh word and four clauses per bit. The quarter-round therefore introduces, per addition, the masks on its two inputs and its output plus the fork masks. The hull enumerator fixes the input and output masks of a given trail, bounds the weight with the totalizer, and after every satisfying assignment adds a blocking clause on all wire-mask variables; the sign of each trail is the product of the local Walsh signs ($-\frac{1}{2}$ when the three-variable majority approximation is used, and at bit 0 when the AND is approximated by $x \oplus y$) and of $(-1)^{\langle m, c \rangle}$ for every constant c XORed into a wire carrying mask m .

D Full two-round ranking of rotation constants

The 21 candidates of §4.3: the 20 best by one-round avalanche among the 1338 choices tied at 1.5-round weight 12, plus the baseline. All satisfy “no two-round trail of weight ≤ 24 ” (`data/diff/screen_r2_top.csv`, `screen_r2_top_k24.csv`); the baseline was run further to ≥ 33 .

998

rank	(r_0, r_1, r_2, r_3)	1.5-round wt	2-round wt	avalanche mean	never-flip pairs (4000 trials)
1	(24,21,16,20)	12	≥ 25	0.3814	6133
2	(24,21,16,23)	12	≥ 25	0.3814	6317
3	(24,21,16,30)	12	≥ 25	0.3813	6059
4	(16,21,24,9)	12	≥ 25	0.3812	6176
5	(16,21,24,24)	12	≥ 25	0.3812	6019
6	(16,21,24,8)	12	≥ 25	0.3812	5985
7	(16,5,8,6)	12	≥ 25	0.3800	7182
8	(16,21,25,22)	12	≥ 25	0.3792	6282
9	(22,17,26,27)	12	≥ 25	0.3790	4070
999	10 (16,25,21,8)	12	≥ 25	0.3787	6870
11	(19,17,25,20)	12	≥ 25	0.3784	5458
12	(27,17,22,31)	12	≥ 25	0.3783	3865
13	(6,14,22,10)	12	≥ 25	0.3777	5622
14	(15,23,27,18)	12	≥ 25	0.3776	8467
15	(18,24,21,15)	12	≥ 25	0.3772	5950
16	(25,17,5,14)	12	≥ 25	0.3771	5425
17	(15,10,8,16)	12	≥ 25	0.3771	6682
18	(16,10,8,18)	12	≥ 25	0.3771	7228
19	(6,17,20,17)	12	≥ 25	0.3770	7126
20	(17,7,13,5)	12	≥ 25	0.3768	8676
–	(16,12,8,7)	12	≥ 33	0.3558	7649

E Reproduction commands

1000

1001 All commands run from the repository root (Windows/clang shown; `c/build.sh` also
 1002 builds on POSIX). Long SAT searches print one line per proven weight level and can be
 1003 resumed from a proven bound with `-kstart`.

```

1004 # implementations and cross-validation (Sec. 5.1)
1005 bash c/build.sh && python tests/check_c_vectors.py && python tests/
1006     check_stream_avx2.py
1007 clang -O3 -std=c11 -mavx2 c/test_avx2.c c/fable_p.c c/fable_p_avx2.c -o
1008     build/test_avx2.exe && ./build/test_avx2.exe
1009
1010 # avalanche (Sec. 4.1)
1011 clang -O3 -std=c11 c/avalanche.c c/fable_p.c -o build/avalanche.exe
1012 for r in 1 2 3 4 5 6; do ./build/avalanche.exe $r 500000 $((1000+r)) data/
1013     avalanche/av_r$r.u32 > data/avalanche/av_r$r.txt; done
1014 python tools/avalanche_report.py
1015
1016 # differential trails (Sec. 4.2)
1017 python sat/fable_diff.py q --rot 16,12,8,7
1018 python sat/fable_diff.py perm --rounds 1 --rot 16,12,8,7
1019 python sat/fable_diff.py perm --rounds 2 --rot 16,12,8,7 --solver cadical153
1020 python sat/fable_diff.py perm --rounds 1 --rot 16,12,8,7 --rate-in
1021 python sat/fable_diff.py perm --rounds 1 --rot 16,12,8,7 --rate-in --rate-
1022     out-nonzero
1023 python sat/fable_diff.py perm --rounds 1 --rot 16,12,8,7 --rate-in --cap-out
1024     -zero --solver cadical153
1025 python sat/fable_diff.py perm --rounds 1 --rot 16,12,8,7 --cap-out-zero --
1026     solver cadical153
1027 python sat/fable_diff.py perm --rounds 0 --half --rot 16,12,8,7 --rate-in
1028     [--cap-out-zero]
1029 python sat/fable_diff.py perm --rounds 2 --rot 16,12,8,7 --rate-in [--cap-
1030     out-zero] --solver cadical153
1031 python sat/fable_diff.py perm --rounds 3 --rot 16,12,8,7 --rate-in --kstart
1032     41 --solver cadical153
1033 python sat/verify_trail.py data/diff/perm_r1_rot16_12_8_7_rate1.json --
1034     samples 20000000
1035
1036 # rotation-constant screening (Sec. 4.3)

```

```

1037 python sat/rot_screen_sat.py --metric q --all --procs 2 --out data/diff/
1038 screen_q_all.csv
1039 python sat/rot_screen_sat.py --metric r1 --all --procs 6 --kmax 10 --out
1040 data/diff/screen_r1_all.csv
1041 python sat/rot_screen_sat.py --metric r1c --from data/diff/cands_r1c.txt --
1042 thresholds 6,9,11 --kmax 16 --procs 8 --out data/diff/screen_r1c_cands.
1043 csv
1044 python tools/av1_tiebreak.py --from data/diff/cands_top12.txt --trials 4000
1045 --out data/diff/av1_tiebreak_top12.csv
1046 python sat/rot_screen_sat.py --metric r2 --from data/diff/cands_r2.txt --
1047 thresholds 16,20 --noexact --procs 8 --out data/diff/screen_r2_top.csv
1048 python sat/rot_screen_sat.py --metric r2 --from data/diff/cands_r2.txt --
1049 thresholds 24 --noexact --procs 8 --out data/diff/screen_r2_top_k24.
1050 csv
1051
1052 # differential-linear biases (Sec. 4.4)
1053 clang -O3 -std=c11 c/dl_bias.c c/fable_p.c -o build/dl_bias.exe
1054 for sr in 1 2 3 4; do ./build/dl_bias.exe $sr 4194304 $((100+sr)) data/dl/
1055 dl_sr${sr}_const.u32; ./build/dl_bias.exe $sr 4194304 $((100+sr)) data/
1056 dl/dl_sr${sr}_noconst.u32 noconst; done
1057 for sr in 3 4; do ./build/dl_bias.exe $sr 4194304 $((200+sr)) data/dl/dl_sr$
1058 {sr}_chacha.u32 chacha 384 512; done
1059 python tools/dl_report.py
1060
1061 # linear trails and hulls (Sec. 4.5)
1062 python sat/fable_lin.py q --rot 16,12,8,7
1063 python sat/fable_lin.py perm --rounds 1 --rot 16,12,8,7
1064 python sat/fable_lin.py perm --rounds 1 --rot 16,12,8,7 --rate-io --solver
1065 cadical153
1066 python sat/fable_lin.py perm --rounds 2 --rot 16,12,8,7 --solver cadical153
1067 python sat/verify_lin.py data/lin/lin_perm_r1_rot16_12_8_7.json
1068 python sat/lin_hull.py enumerate --trail data/lin/lin_perm_r1_rot16_12_8_7.
1069 json --wmax 6
1070 python sat/lin_hull.py find --rounds 2 --fix-in data/lin/
1071 lin_perm_r1_rot16_12_8_7.json
1072 python sat/lin_hull.py enumerate --trail data/lin/lin_perm_r2_fixin.json --
1073 wmax 60 --samples 0
1074
1075 # rotational-XOR, constants, slide, fixed points (Sec. 4.6, 4.7)
1076 python tools/rx_check.py
1077 python tools/check_constants.py
1078
1079 # performance (Sec. 5)
1080 clang -O3 -std=c11 -mavx2 -DFABLE_HAVE_AVX2 c/bench.c c/fable_p.c c/
1081 fable_p_avx2.c -o build/bench.exe && ./build/bench.exe 2000000
1082 clang -O3 -std=c11 -mavx2 c/bench_stream.c c/fable_stream_avx2.c c/
1083 fable_p_avx2.c c/fable_p.c c/fable_aead.c -o build/bench_stream.exe &&
1084 ./build/bench_stream.exe 1024 65536 3 1,4,8
1085 python tools/bench_chacha.py 1024 ; openssl speed -evp chacha20-poly1305 -
1086 seconds 2

```